# Applying Zero-Trust Principles to Kafka: A Performance and Security Trade-off Analysis

Nick Efe Oni

*Supervisor:* Mahshid Delavar

*A report submitted in fulfilment of the requirements for the degree of* MEng in Software Engineering

*in the*

School of Computer Science

April 19, 2026

## Declaration

All sentences or passages quoted in this report from other people's work have been specifically acknowledged by clear cross-referencing to author, work and page(s). Any illustrations that are not the work of the author of this report have been used with the explicit permission of the originator and are specifically acknowledged. I understand that failure to do this amounts to plagiarism and will be considered grounds for failure in this project and the degree examination as a whole.

Name: Nick Efe Oni

Signature: Nick Efe Oni

Date: April 19, 2026

# Abstract

an abstract is provided to give a brief overview and summary of the core topics, arguments, and findings . Because it must summarize your entire project—including your final benchmarking results from Chapter 5 and your final conclusions from Chapter 7—you should write the abstract at the very end of the process, once the rest of your dissertation is completely finished.

# Contents

**1 Introduction** **1**

**2 Literature Review** **3**
2.1 Introduction to Literature Review . . . . . 3
2.2 Apache Kafka & Distributed Streaming Systems . . . . . 4
2.3 Performance in Kafka Systems . . . . . 5
2.4 Security Mechanisms in Kafka . . . . . 7
2.5 Zero Trust Architecture . . . . . 8
2.6 Zero Trust in Distributed Streaming Systems . . . . . 9
2.7 TLS/mTLS Performance Overhead . . . . . 11
2.8 Benchmarking Methodologies . . . . . 13
2.9 Synthesis & Research Gap . . . . . 13

**3 Design, Implementation, and Testing** **14**

**4 Results and Discussion** **15**

**5 Evaluation, Conclusions, and Further Work** **16**

**Appendices** **19**

**A An Appendix of Some Kind** **20**

**B Ignore - Templatey Stuff** **21**
B.1 Section Template Graph . . . . . 21
B.2 Section Template Comparison . . . . . 22
B.3 Summary . . . . . 22

**C Another Appendix** **23**

# List of Figures

2.1 Simplified Apache Kafka architecture showing producer applications publishing events to cluster of brokers, and consumer applications pulling events from the cluster. . . . . 5
2.2 Comparison of Kafka's native zero-copy plaintext path and the TLS/mTLS path. In plaintext mode, Kafka can use sendfile() to transfer data efficiently from the page cache to the socket. Under TLS/mTLS, this optimisation is disabled, forcing data through user-space for encryption before transmission. 7
2.3 Kafka security data path illustrating how TLS/mTLS, certificate-based identity verification, principal mapping, and authorisation checks are applied across client-broker and broker-to-broker communication. . . . . 8
2.4 Conceptual Zero Trust control model for a distributed streaming pipeline, illustrating how mTLS, policy enforcement, least-privilege access, and continuous monitoring are applied across producer, broker, stream-processing, and consumer components within a segmented trust boundary. . . . . 11

B.1 A simple figure in LaTeX. Reproduced from http://tinyurl.com/nqtrlj5 with the permission of the copyright owner. . . . . 22

# List of Tables

B.1 A simple table in LaTeX. . . . . 22

# Chapter 1

# Introduction

With the evolution of distributed systems [1], event-streaming platforms such as Apache Kafka have been used to connect hundreds of applications in the cloud [2–4], as critical aspects shifted to data consistency, performance optimisation, fault tolerance, and security [5, 6]. Apache Kafka (or Kafka in short) is a popular open-source distributed event-streaming platform [6, 7], known for its performance, low latency, fault tolerance, and high throughput – capable of handling millions of messages per second [3, 4]. These properties make it a critical component in modern data-driven architectures. However, this shift has increased the number of insider threats, as traditional network security was not designed against attacks from within the network [1, 2, 8, 9]. This led industries to increasingly adopt the zero-trust security paradigm, whose core idea means "never trust, always verify" [1, 2, 8–10]. Its implementation includes mutual authentication (mTLS), least-privilege access, continuous monitoring, and data-centric security, which are key to establishing comprehensive protection across distributed environments [1, 2, 8–10]. Hence, while Zero Trust Architecture is typically implemented as a system-level security paradigm rather than within a single platform, Apache Kafka provides several native mechanisms that align with key Zero Trust principles, particularly in transport security and authentication [1, 8, 9, 11–14].

Despite existing research providing strong theoretical and practical foundations for zero trust in streaming systems [1, 2, 15] and Kafka widely analysed for its properties [3, 7, 16], there is no unified framework and sufficiently systematic empirical evaluation of the performance impact of zero-trust-aligned mechanisms in Kafka-based environments, particularly those related to transport security such as TLS and mTLS [2, 3, 14]. For example, multiple sources identify TLS and mTLS as central mechanisms for enforcing secure communication and mutual authentication in distributed systems [1, 2, 8, 9, 11–13, 17]. However, they introduce significant performance overhead and latency [17–19], which is reinforced by Apache Kafka's official documentation stating that enabling TLS/SSL forces cryptographic operations into user space, disabling Kafka's highly efficient sendfile zero-copy optimisation [11, 18, 20]. However, the scale of this degradation is highly context-dependent and remains insufficiently quantified across controlled Kafka configurations.

Therefore, the research question addressed in this dissertation is:

"What is the performance overhead introduced by zero-trust-aligned security mechanisms, particularly mTLS, in high-throughput Kafka-based distributed systems?"

To answer this research question, this dissertation designs, implements, and evaluates a configurable Kafka benchmarking artefact for measuring the performance cost of transport security in controlled cloud-based Kafka deployments. The artefact provisions Apache Kafka clusters on AWS EC2 using Terraform, deploys Kafka in KRaft mode, and supports three security configurations: plaintext, TLS, and mutual TLS (mTLS). Benchmark execution is driven by structured configuration files so that equivalent workloads can be repeated across security modes and compared under matched conditions.

The evaluation methodology combines a producer-side factorial benchmark campaign with a smaller targeted consumer-side validation study. The producer campaign varies operational parameters such as broker count, replication factor, minimum in-sync replicas, message size, target throughput, producer count, acknowledgements, batch size, and compression type. This allows the artefact to examine not only whether TLS and mTLS introduce overhead, but also how that overhead changes under different Kafka workload and durability settings. The consumer-side study is included to validate whether secure transport also affects Kafka's read path, particularly where mTLS processing interacts with larger messages and compressed payloads.

For each benchmark run, the artefact records application-level metrics, including throughput and latency, alongside host-level telemetry from the benchmark client and Kafka brokers, including CPU, memory, disk, and network activity. Raw benchmark logs are parsed into structured JSON and CSV outputs, allowing reproducible comparison of plaintext, TLS, and mTLS results. This approach enables the dissertation to empirically quantify the trade-offs introduced by Kafka transport-security mechanisms and to relate observed latency or throughput degradation to underlying resource utilisation.

# Chapter 2

# Literature Review

## 2.1 Introduction to Literature Review

This literature review explores the application of modern security paradigms in distributed messaging systems, specifically focusing on the implementation of Zero-Trust-aligned security mechanisms in Apache Kafka. It provides the foundation for the research gap identified in Chapter 1: the lack of systematic empirical evaluation of the performance impact of Zero-Trust-aligned transport-security mechanisms, particularly TLS and mTLS, in high-throughput Kafka environments.

The review is organised into a sequence of thematic sections. It begins by examining the architecture and operational characteristics of Apache Kafka, followed by an analysis of its performance behaviour and the architectural factors that enable high throughput and low latency. It then evaluates Kafka's native security mechanisms, focusing on how enabling these controls alters system performance.

The discussion subsequently expands to the broader principles of Zero Trust Architecture and the challenges of applying these principles to distributed and streaming systems. This includes critical analysis of access control models, continuous authentication, and cryptographic enforcement in high-throughput environments. The review then focuses specifically on TLS and mTLS, synthesising existing literature on their computational and network overhead in distributed systems.

Finally, the review examines benchmarking methodologies for distributed systems, highlighting the need for controlled, reproducible experimentation. This progression establishes the key gap addressed in this dissertation: while TLS and mTLS are widely recognised as essential Zero-Trust-aligned mechanisms, their precise performance impact within Kafka-based streaming systems remains insufficiently quantified under realistic and controlled configurations.

## 2.2 Apache Kafka & Distributed Streaming Systems

Distributed streaming systems, such as Apache Kafka, are engineered to be fault-tolerant [6, 16, 20], have high-availability [6, 16], and handle the storage and parallel processing of massive streams of data [6, 16, 20], while ensuring data consistency and reliability [3, 16].

Apache Kafka consists of servers (brokers) and clients (producers/consumers) that communicate via a high-performance TCP network protocol, which contributes to Kafka's performance [6, 16]. Producers are client applications that publish (write) events to brokers, whereas consumers are those that subscribe to (read and process) these events by pulling from the brokers [6, 16]. Thus, brokers act as an intermediary cluster of servers between producers and consumers [6, 16]. A simple illustration of this can be found in Figure 2.1.

These decoupling and pull-based consumption mechanisms contribute to Apache Kafka's fault tolerance and high throughput [6, 20]. Consumer applications can fetch large data batches at their own maximum processing capacity without being overwhelmed by the data production rate [20].

Moreover, events are organised and durably stored in "topics", which are implemented as append-only, immutable, time-ordered logs on the disk [3, 6, 20]. These topics are subdivided into partitions distributed across multiple brokers, enabling massive horizontal scalability and the storage and parallel processing of massive streams of data [6, 16, 20].

Despite the benefits, there are also trade-offs and drawbacks. Kafka's performance is affected by batching, acknowledgement semantics, and broker-level thread and quota configuration, which introduce explicit throughput-latency-durability trade-offs [3, 11, 12, 20]. In addition, maintaining consistency across replicas and coordinating partition leadership can impose non-trivial overhead, particularly under sustained high-throughput workloads and inappropriate configurations [3, 11, 20].

In essence, Apache Kafka is a widely adopted distributed streaming system with multiple architectural features and mechanisms designed to support data consistency, reliability, low latency, and high throughput [3, 6, 16, 20]. However, these benefits come with trade-offs and complexities, as Apache Kafka's performance emerges not solely from its architectural features and mechanisms, but also from the selected configuration, coordination costs and the deployment environment [3, 11–13, 20]. This has important implications for the integration of security protocols such as TLS and mTLS, which introduce additional communication and computational overhead and therefore necessitate careful evaluation within Kafka's distributed architecture [11–13, 18].

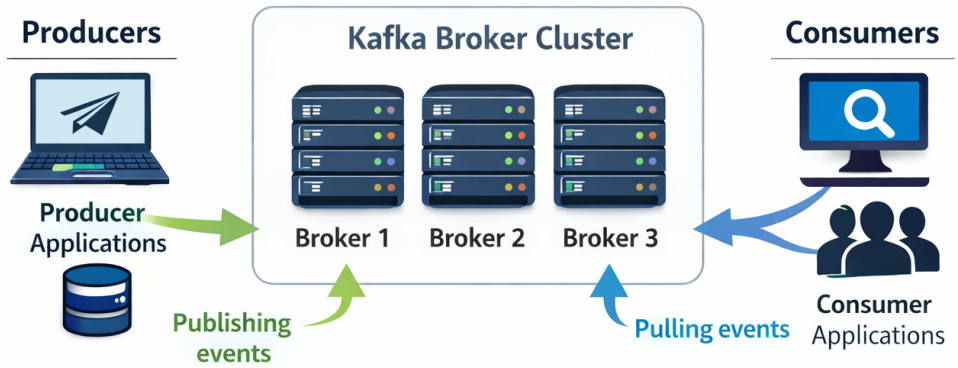


Figure 2.1: Simplified Apache Kafka architecture showing producer applications publishing events to cluster of brokers, and consumer applications pulling events from the cluster.

## 2.3 Performance in Kafka Systems

The performance of Apache Kafka arises from key architectural choices. For example, bypassing traditional user-space memory management and $O(\log N)$ random-access data structures, such as B-Trees, in favour of simple constant-time $O(1)$ append-only logs [3,6,20]. Furthermore, it heavily relies on the operating system's page cache and aggressively employs zero-copy I/O mechanisms, specifically the 'sendfile' system call, to transfer data directly from the kernel to the network socket [20]. By offloading memory management to the OS and using $O(1)$ sequential disk operations, Kafka maintains maximum performance and avoids unnecessary data movements [6, 16, 20]. This dependence on OS-level optimisations makes the system incredibly fast, but also highly sensitive to user-space interventions introduced by security mechanisms such as TLS and mTLS [11,17–20]. To better visualise this contrast, see figure 2.2.

Additionally, Kafka's speed is driven by data-flow mechanisms such as parallel partitioning, aggressive batching, and decoupled pull-based consumption [3, 6, 16, 20]. Topics are divided into partitions spread across multiple brokers, enabling massive horizontal scalability by

allowing client applications to read and write data in parallel [6, 16, 20]. Producers amortise network round-trip overhead through asynchronous sends, utilising specific configurations to group individual events into dense, compressed payloads [3, 12, 16, 20]. Concurrently, consumers operate on a pull-based model utilising lightweight offset tracking, allowing them to fetch these large, compressed batches at their own maximum capacity without overwhelming the broker [6,7,13,16,20]. Due to these mechanisms, the computational burden of decompression shifts to the consumer, drastically reducing network bandwidth saturation and the CPU load of the brokers [3, 12, 13, 16, 20]. Consequently, Kafka's native performance heavily relies on keeping these network payloads perfectly tuned and lightweight, conditions that are fundamentally disrupted when complex user-space cryptography is introduced to the process [11–13, 17–20].

Moreover, Kafka configuration choices introduce explicit throughput-latency-durability trade-offs [3, 11–13, 16, 20]. Producers can be configured to require full consensus from all in-sync replicas or push events asynchronously [3, 12, 16, 20]. Although the former guarantees durability, it significantly increases latency and decreases throughput, thus degrading raw performance [3, 11, 12, 16, 20]. Similarly, enforcing Exactly-Once Semantics (EOS) to prevent duplicate messages further increases latency by requiring strict state tracking and transactional coordination across the cluster [3, 11, 12, 16, 20]. Overall, these trade-offs demonstrate that Apache Kafka's performance is not a static absolute, but a highly configurable spectrum in which the overhead of mandatory operational constraints, such as those required for highly secure Zero-Trust environments, must be carefully benchmarked [3, 11–13, 17–20].

Furthermore, according to a queueing-theoretic model applied to publish-subscribe systems, the average latency ($L$) of a message is mathematically defined by the processing time ($\tau$) and system utilisation ($U$) as:

$$L = \frac{\tau}{1 - U} \tag{2.1}$$

[7, 16, 19]. Any configuration or architectural shift that increases processing time ($\tau$) pushes CPU utilisation ($U$) closer to 1.0, causing latency to grow non-linearly rather than linearly as the system approaches saturation [3, 7, 16, 19]. This theoretical relationship provides a foundation for understanding the identified research gap: because Zero Trust mechanisms such as Mutual TLS (mTLS) force operations into user-space, their performance overhead cannot be assumed to be linear, necessitating rigorous, context-specific empirical evaluation in high-throughput Kafka systems [3, 7, 11, 17–20]. This is because, by forcing operations into user-space, it effectively disables or limits zero-copy optimisations and significantly increases $\tau$ via complex cryptography [7, 11, 17–20].

Ultimately, Kafka's performance, while capable of achieving high throughput, is highly sensitive to both internal client architecture and the underlying realities of system

utilisation [3, 7, 11–13, 19]. This highlights the need to examine how security mechanisms are implemented within Kafka systems, particularly those that introduce additional computational and communication overhead. Understanding these mechanisms is therefore essential before evaluating their impact on performance in high-throughput environments [3, 11–13, 17–20].

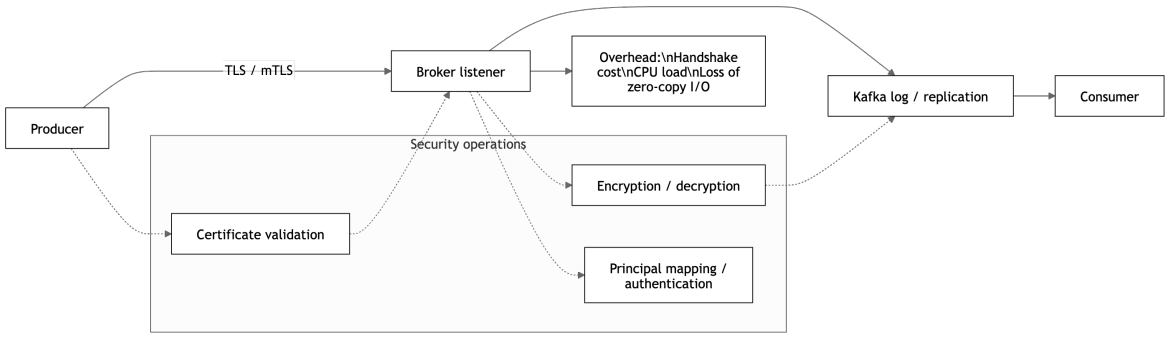


Figure 2.2: Comparison of Kafka's native zero-copy plaintext path and the TLS/mTLS path. In plaintext mode, Kafka can use sendfile() to transfer data efficiently from the page cache to the socket. Under TLS/mTLS, this optimisation is disabled, forcing data through user-space for encryption before transmission.

## 2.4 Security Mechanisms in Kafka

To protect sensitive data streams against both external and internal threats, Kafka provides a native security toolkit that includes encryption, authorisation, and authentication [6, 11–14]. However, while these controls successfully establish a secure boundary, their activation fundamentally alters the mechanics of Kafka's data flow, introducing systemic complexities that directly conflict with the platform's baseline performance design [11–13, 17–20]. These gaps directly motivate the research presented in this dissertation.

For instance, encryption in transit protects the confidentiality of data payloads across distributed environments, preventing eavesdropping and Man-in-the-Middle attacks [1, 9, 11–14, 17]. This is implemented through Transport Layer Security (TLS) via specific listener protocols, allowing administrators to precisely dictate cryptographic strength using security configurations such as '*ssl.enabled.protocols*' and '*ssl.cipher.suites*' [11–14,17]. By enabling these security configurations, Apache Kafka's throughput efficiency significantly degrades due to a compounding bottleneck [3, 11, 14, 17, 18, 20]. This bottleneck emerges from the heavy mathematical overhead of encrypting and decrypting every payload, and the loss of zero-copy I/O optimisations, since TLS/SSL libraries are required to operate in user space by design [11, 17–20]. Despite the literature possessing a clear critique regarding this specific architectural clash and the analysis of TLS overhead in traditional stateless

web servers, empirical research explicitly quantifying this degradation within Kafka and its specific configurations remains sparse [3, 11, 14, 17–20].

Alternatively, there is mTLS, which is enforced through the broker's *ssl.client.auth* configuration, allowing the use of cryptographically signed $X.509$ certificates to verify identities between brokers, consumers, and producers [11–14, 17]. For a simple illustration of this enforcement, at the end of this chapter, see Figure 2.3. However, because Apache Kafka's distributed design relies heavily on internal replication, mTLS may also be applied to all internal broker-to-broker traffic, substantially increasing the cryptographic workload across the entire system and significantly reducing performance [9, 11, 17–20]. Moreover, if thousands of clients attempt to reconnect simultaneously, repeated TLS/mTLS handshakes create an authentication surge that consumes broker resources and interferes with data-plane performance [1, 2, 11, 17–20].

In conclusion, while Apache Kafka provides robust optional native security measures, enabling them causes heavy computational costs that are insufficiently explored across the literature, as evidenced by the sparse, context-limited, and inconsistently reported empirical evaluations of Kafka-specific configurations and their associated trade-offs [3, 7, 11–14, 17–20]. Among these measures, transport security (TLS and mTLS) is particularly significant, as it directly affects the data path and introduces measurable computational overhead during communication [11–13, 17, 18, 20].

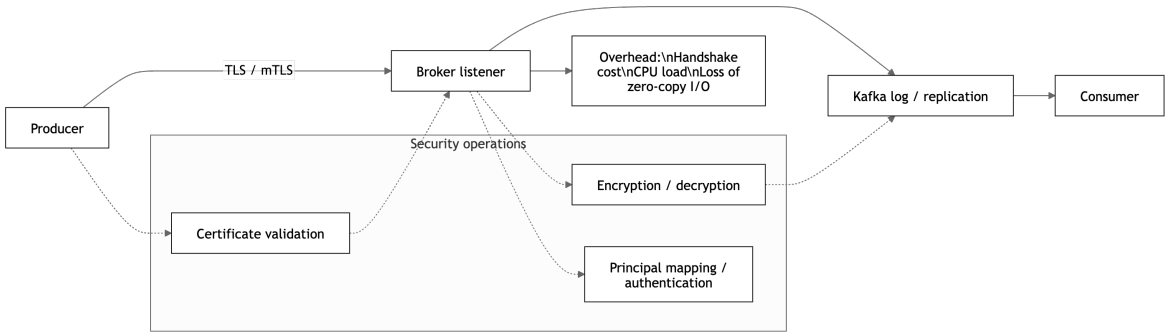


Figure 2.3: Kafka security data path illustrating how TLS/mTLS, certificate-based identity verification, principal mapping, and authorisation checks are applied across client-broker and broker-to-broker communication.

## 2.5 Zero Trust Architecture

The limitations of native security mechanisms in distributed environments highlight the necessity for a more comprehensive security paradigm, leading to the widespread adoption of Zero Trust Architecture (ZTA) [1, 2, 5, 8, 9]. Unlike traditional perimeter-based models

that assume internal networks are secure, ZTA operates on the foundational principle of "never trust, always verify," eliminating implicit trust entirely [1, 2, 8–10]. Instead, security is maintained through continuous verification, the enforcement of least-privilege access, and strict identity-based control [1, 2, 8–10]. This paradigm shift ensures that security is proactively applied at every interaction rather than merely at the network boundary, an approach that is increasingly critical in modern, highly decentralised cloud environments [1, 2, 8, 9].

To enforce this continuous verification, ZTA relies on a combination of dynamic policies and structural components rather than a single monolithic technology [1, 8–10]. At its core, ZTA necessitates context-aware authentication and fine-grained access control models, frequently implemented through logical components such as Policy Decision Points (PDP) and Policy Enforcement Points (PEP) [1, 8–10]. Furthermore, a fundamental requirement for establishing these secure, segmented interactions is the mandatory encryption of all communication channels [1, 8–10]. Consequently, cryptographic protocols such as TLS and mTLS become foundational mechanisms in many Zero Trust implementations rather than optional add-ons [1, 8, 9, 17].

While ZTA provides robust protection against both external and insider threats, analytical studies reveal that its implementation introduces significant operational complexities [1, 8–10]. Transitioning to a Zero-Trust model requires major infrastructure restructuring, complex identity management, and continuous monitoring, making the paradigm resource-intensive [1, 2, 5, 8–10]. This introduces a critical architectural tension: the strict, continuous cryptographic verification demanded by Zero Trust consumes substantial computational resources, threatening to disrupt the delicate performance balance of latency-sensitive systems [1, 7–9, 17–20].

## 2.6 Zero Trust in Distributed Streaming Systems

The challenges in applying the principles of Zero Trust Architecture (ZTA) also apply to distributed streaming systems [1, 2, 5, 8, 9]. Despite modern event-streaming pipelines being designed and used between remote edge devices, multi-cloud environments, and third-party networks, applying ZTA can conflict with their low-latency, high-throughput architecture by introducing continuous authentication, encryption, and segmentation overhead [2, 5, 8, 9]. In these environments, data flows continuously, and every component must be treated as untrusted until explicitly verified, while maintaining high throughput and low latency [1, 2, 5, 8, 9]. Consequently, the ZTA mandate of continuous verification and mutual authentication must be applied dynamically to machine-to-machine communications throughout the pipeline [1, 2, 8, 9, 17]. See Figure 2.4 to see how Zero Trust controls are distributed across a streaming pipeline. However, the literature reveals significant conflicting findings and practical limitations regarding how these mechanisms can be

implemented without compromising the throughput and latency requirements of streaming data systems [2, 17–19].

A primary area of debate in the literature concerns the implementation of dynamic access control models [1, 2, 8–10]. To enforce the Zero-Trust principle of least privilege, organisations typically adopt Role-Based Access Control (RBAC) or Attribute-Based Access Control (ABAC) [1, 2, 8, 9]. Although RBAC is widely implemented due to its simplicity, researchers heavily criticise it for lacking the granularity required in dynamic streaming environments, noting that it frequently leads to a "role explosion" where roles and privileges accumulate insecurely over time [1, 2, 8, 9]. In contrast, ABAC provides finer-grained access control by evaluating contextual attributes such as environmental conditions, device state, time, location, and identity metadata [1, 2, 9]. However, defining ABAC policies is highly prone to rule conflicts, and evaluating complex contextual attributes at run-time introduces explicit computational latency [1, 2, 8, 9]. In real-time data streaming (where millions of events are processed per second), this run-time policy evaluation creates a severe bottleneck, highlighting a conflict between the theoretical necessity for fine-grained authorisation and the need for constant-time, low-latency processing [1, 2, 9].

Another area of dispute concerns the operational constraints of continuous Mutual TLS (mTLS) authentication [2, 17, 18]. Some empirical network measurements suggest that with modern protocols such as TLS 1.3 and adequately provisioned hardware, the steady-state run-time overhead of encrypted connections is practically negligible after the initial handshake [14, 19]. However, researchers focusing on distributed and edge-to-cloud streaming pipelines strongly disagree with this generalisation [17, 18]. In realistic streaming architectures involving large volumes of small message batches, heterogeneous IoT endpoints, and resource-constrained edge devices, the heavy mathematical complexity of repeated authentication handshakes and continuous encrypted transport becomes a severe limitation [1, 2, 17–19]. The recurring computational cost of certificate management, key rotation, and payload encryption inherently clashes with the strict latency and throughput constraints of real-time systems [1, 2, 17–19].

Ultimately, integrating ZTA into distributed streaming systems presents an unresolved architectural paradox [1, 2, 9, 17, 18]. While the industry universally acknowledges that traditional perimeter defences are insufficient for modern data pipelines, the current literature indicates that implementing RBAC, granular ABAC policies, micro-segmentation, and continuous mTLS introduces severe computational and architectural bottlenecks [1, 2, 9, 17–19]. This establishes a critical gap: the strict, continuous verification principles of ZTA fundamentally clash with the constant-time, zero-copy optimisations required by platforms like Apache Kafka, demanding rigorous empirical investigation to evaluate these heavy security mandates against the necessity for extreme data velocity [1–3, 9, 17–19].

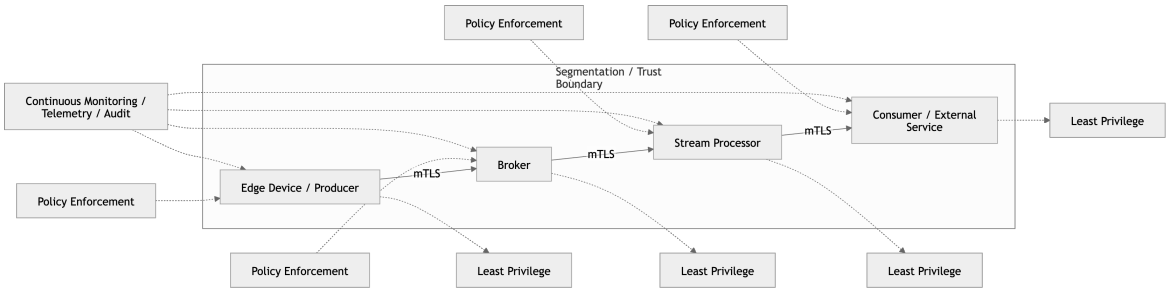


Figure 2.4: Conceptual Zero Trust control model for a distributed streaming pipeline, illustrating how mTLS, policy enforcement, least-privilege access, and continuous monitoring are applied across producer, broker, stream-processing, and consumer components within a segmented trust boundary.

## 2.7 TLS/mTLS Performance Overhead

To quantify the operational constraint imposed by encrypted data streams, empirical literature evaluates the computational overhead of Transport Layer Security (TLS) within the context of traditional, stateless web servers. These studies systematically partition the performance cost of secure channels into the asymmetric public-key connection setup and the symmetric bulk cipher data exchange, measuring total throughput degradation against plaintext baselines. The literature consistently finds that establishing a TLS connection imposes an immense computational burden, resulting in an overall throughput degradation factor ranging from 3.4 to 9. Specifically, the asymmetric cryptographic operations are identified as the single most expensive protocol component, consuming up to 58% of the total CPU processing time during connection establishment according to a few researchers. However, these evaluations are fundamentally limited when applied to event-streaming platforms because they model workloads where a new, ephemeral TCP connection is established for nearly every request, and where data is already processed entirely within user-space memory. In contrast, Apache Kafka utilises persistent connections and achieves its native velocity through zero-copy I/O mechanisms, specifically the ‘sendfile‘ system call. Consequently, traditional web server benchmarks fall short because they fail to capture the compounding architectural penalty specific to Kafka: the total loss of zero-copy optimizations, which drastically shifts the baseline network performance before the mathematical cost of cryptography is even applied. This indicates that evaluating TLS overhead in Kafka requires benchmarking under persistent, high-throughput workloads that preserve long-lived connections and capture system-level data movement effects.

To assess cryptographic costs outside of ephemeral connections, a distinct body of literature evaluates TLS latency within persistent communication models, focusing primarily on edge-computing and task-offloading scenarios. These large-scale end-to-end measurement

campaigns analyse the Flow Completion Time (FCT) of encrypted payloads over continuous, full-duplex channels to isolate steady-state cryptographic costs from handshake penalties. The findings empirically demonstrate that utilising persistent connections effectively masks the initial TLS handshake latency, allowing the congestion window to reach steady-state and reducing the symmetric encryption application-level delay to a nearly negligible margin. While this accurately highlights the computational benefits of session persistence, these results are strictly limited because they evaluate isolated, single client-to-server interactions over asymmetric access links. This topological assumption falls short for Apache Kafka, which operates as a heavily replicated distributed log. When mTLS is enabled in Apache Kafka, the symmetric encryption penalty is not isolated to a single edge connection, but rather geometrically multiplied across the entire cluster. Existing edge-computing experiments are limited because they omit this massive, synchronised cryptographic multiplier effect, as well as the compounding CPU saturation that occurs when a broker must manage symmetric encryption alongside end-to-end batch decompression. Therefore, evaluation in Kafka must account for intra-cluster replication and distributed processing, where cryptographic overhead is applied across multiple coordinated nodes rather than a single communication channel.

However, the literature does include recent systematic reviews that analyse the benchmarking practices explicitly utilised for performance evaluation on event-streaming architectures, such as Apache Kafka. These studies evaluate how empirical performance results are obtained across standard suites (e.g., TPCx-Kafka), synthetic frameworks, and custom domain-specific workloads. The literature reveals a systemic reproducibility crisis within the streaming domain, with one particular source finding that 62% of the surveyed empirical studies frequently omit foundational configuration variables from their reported methodology. Crucial parameters that directly govern the cryptographic workload, such as partition counts, replication factors, and precise batch sizes, are routinely left undisclosed. This reporting failure renders the existing performance impact on Apache Kafka due to added security measures literature highly limited. Failing to standardise and publish these parameters makes cross-study comparison of cryptographic overhead impossible. Existing Apache Kafka experiments fall short because they fail to systematically isolate the cryptographic penalty from general system saturation under strictly controlled baseline conditions, directly establishing the absolute necessity for an automated, deterministic benchmarking framework to quantify the true cost of zero-trust security. This highlights the need for controlled, configuration-explicit benchmarking frameworks that isolate individual system variables and enable reproducible comparison across security configurations. Yet, one thing that is clear is that the performance impact of mTLS in Kafka is not a static absolute; it is a highly variable spectrum dictated by how efficiently heavy mathematical costs can be horizontally parallelised across partitions or amortised across dense message batches.

## 2.8 Benchmarking Methodologies

## 2.9 Synthesis & Research Gap

Although existing studies establish that TLS/mTLS and Zero-Trust-aligned mechanisms introduce computational and communication overhead, their empirical relevance to Kafka remains limited [**?**, 2, 18, 19]. Generic TLS studies typically evaluate stateless web servers or persistent client-server connections, rather than Kafka's partitioned, replicated, batch-oriented data path [18–20]. Conversely, Kafka benchmarking studies often focus on throughput, batching, replication, or exactly-once semantics without systematically isolating the cost of security mechanisms such as TLS, mTLS, cipher-suite selection, client authentication, and broker-to-broker encryption [3, 11, 20]. Where Kafka security is discussed, evaluations are often domain-specific, incompletely configured, or insufficiently reproducible [3,14]. As a result, the literature does not yet provide a controlled, Kafka-specific empirical account of how Zero-Trust-aligned security mechanisms interact with Kafka's native performance optimisations under realistic high-throughput workloads [2,3,20]. Consequently, this dissertation focuses primarily on the performance implications of TLS and mTLS within Kafka deployments and on the need for controlled, reproducible benchmarking to quantify their performance impact.

# Chapter 3

# Design, Implementation, and Testing

This is where you explain how you are going to solve the problem. To get top marks, you must provide an in-depth analysis of the problem area, outline clear project requirements, and present a precise, methodical design. For your project, this will likely be your methodology chapter outlining your Kafka testing environment. You must thoroughly describe how you built your experimental system and how you conducted your tests. The rubric specifically looks for a comprehensive discussion of your testing processes

# Chapter 4

# Results and Discussion

This is where you present your findings (e.g., your throughput and latency data). To reach the 70+ band, your results must be presented in a logical order alongside a "perceptive and critical discussion" of what the data actually means

# Chapter 5

# Evaluation, Conclusions, and Further Work

Your final chapter must evaluate how well you satisfied your original objectives, provide an insightful summary of your conclusions, and discuss appropriate directions for future research

# Bibliography


[1] H. Kang, G. Liu, Q. Wang, L. Meng, and J. Liu, "Theory and application of zero trust security: A brief survey," *Entropy*, vol. 25, no. 12, p. 1595, 2023.

[2] H. Bhoite, "Zero-trust architecture in streaming dataflows," https://www.authorea.com/doi/full/10.36227/techrxiv.175303721.12297807, accessed: 2026-03-26.

[3] M. Mohammad, "Analysis of design patterns and benchmark practices in apache kafka event-streaming systems," in *2025 5th International Conference on Intelligent Cybernetics Technology & Applications (ICICyTA)*, 2025, pp. 612–617.

[4] "Powered by," https://kafka.apache.org/powered-by/, accessed: 2026-03-24.

[5] A. Kumar, "Distributed systems in modern enterprise architecture: Challenges and solutions," *IJSAT*, vol. 16, no. 1, p. 2447, 2025.

[6] "Introduction," https://kafka.apache.org/42/getting-started/introduction/, accessed: 2026-03-24.

[7] A. Sundbom, "Parallel kafka producer applications: Their performance and its limitations," Master's thesis, KTH Royal Institute of Technology, 2023.

[8] V. A. Vora, "Demystifying zero trust security: The no-trust network paradigm," *Journal of Computer Science and Technology Studies*, vol. 7, no. 3, pp. 141–148, 2025.

[9] N. F. Syed, S. W. Shah, A. Shaghaghi, A. Anwar, Z. Baig, and R. Doss, "Zero trust architecture (zta): A comprehensive survey," *IEEE Access*, vol. 10, pp. 57 143–57 179, 2022.

[10] M. Tsai, S. Lee, and S. W. Shieh, "Strategy for implementing of zero trust architecture," *IEEE Transactions on Reliability*, vol. 73, no. 1, pp. 93–100, 2024.

[11] "Broker configs," https://kafka.apache.org/42/configuration/broker-configs/, accessed: 2026-03-26.

[12] "Producer configs," https://kafka.apache.org/42/configuration/producer-configs/, accessed: 2026-03-26.

[13] "Consumer configs," https://kafka.apache.org/42/configuration/consumer-configs/, accessed: 2026-03-26.

[14] J. Meka, "Financial services cloud transformation: Securing sensitive data in kafka event streams," *Journal of Computer Science and Technology Studies*, vol. 7, no. 4, pp. 1023–1028, 2025.

[15] A. A. Süzen and O. Ceylan, "The advantages and implementation challenges within the scope of the basic principles of transition to zero trust architecture," *International Journal of 3D Printing Technologies and Digital Industry*, vol. 8, no. 3, pp. 416–427, 2024.

[16] T. P. Raptis and A. Passarella, "A survey on networked data streaming with apache kafka," *IEEE Access*, vol. 11, pp. 85 333–85 350, 2023.

[17] C. H. Jack, S. K. Teck, L. T. Ming, and D. Y. Hong, "An overview analysis of authentication mechanism in microservices-based software architecture: A discussion paper," in *2023 IEEE 8th International Conference on Software Engineering and Computer Systems (ICSECS)*, 2023, pp. 1–6.

[18] C. Coarfa, P. Druschel, and D. S. Wallach, "Performance analysis of tls web servers," *ACM Transactions on Computer Systems*, vol. 24, no. 1, pp. 39–69, 2006.

[19] M. Iorio, F. Risso, and C. Casetti, "When latency matters: Measurements and lessons learned," *SIGCOMM Computer Communication Review*, vol. 51, no. 4, pp. 2–13, 2021.

[20] "Design," https://kafka.apache.org/42/design/design/, accessed: 2026-03-24.

# Appendices

# Appendix A

# An Appendix of Some Kind

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. Nulla consequat massa quis enim. Donec pede justo, fringilla vel, aliquet nec, vulputate eget, arcu. In enim justo, rhoncus ut, imperdiet a, venenatis vitae, justo. Nullam dictum felis eu pede mollis pretium. Integer tincidunt. Cras dapibus. Vivamus elementum semper nisi. Aenean vulputate eleifend tellus. Aenean leo ligula, porttitor eu, consequat vitae, eleifend ac, enim. Aliquam lorem ante, dapibus in, viverra quis, feugiat a, tellus. Phasellus viverra nulla ut metus varius laoreet. Quisque rutrum. Aenean imperdiet. Etiam ultricies nisi vel augue. Curabitur ullamcorper ultricies nisi. Nam eget dui. Etiam rhoncus. Maecenas tempus, tellus eget condimentum rhoncus, sem quam semper libero, sit amet adipiscing sem neque sed ipsum. Nam quam nunc, blandit vel, luctus pulvinar, hendrerit id, lorem. Maecenas nec odio et ante tincidunt tempus. Donec vitae sapien ut libero venenatis faucibus. Nullam quis ante. Etiam sit amet orci eget eros faucibus tincidunt. Duis leo. Sed fringilla mauris sit amet nibh. Donec sodales sagittis magna. Sed consequat, leo eget bibendum sodales, augue velit cursus nunc.

# Appendix B

# Ignore - Templatey Stuff

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. Nulla consequat massa quis enim. Donec pede justo, fringilla vel, aliquet nec, vulputate eget, arcu. In enim justo, rhoncus ut, imperdiet a, venenatis vitae, justo. Nullam dictum felis eu pede mollis pretium. Integer tincidunt. Cras dapibus. Vivamus elementum semper nisi. Aenean vulputate eleifend tellus. Aenean leo ligula, porttitor eu, consequat vitae, eleifend ac, enim. [**?**] Aliquam lorem ante, dapibus in, viverra quis, feugiat a, tellus. Phasellus viverra nulla ut metus varius laoreet. Quisque rutrum.

$$M = \frac{1}{T}\sum_{t=1}^{T} e(t)/\max_{t}[e(t)] \tag{B.1}$$

Aenean imperdiet. Etiam ultricies nisi vel augue. Curabitur ullamcorper ultricies nisi. Nam eget dui. Etiam rhoncus. Maecenas tempus, tellus eget condimentum rhoncus, sem quam semper libero, sit amet adipiscing sem neque sed ipsum. Nam quam nunc, blandit vel, luctus pulvinar, hendrerit id, lorem. Maecenas nec odio et ante tincidunt tempus. Donec vitae sapien ut libero venenatis faucibus. Nullam quis ante. Etiam sit amet orci eget eros faucibus tincidunt. Duis leo. Sed fringilla mauris sit amet nibh. Donec sodales sagittis magna. Sed consequat, leo eget bibendum sodales, augue velit cursus nunc.

## B.1 Section Template Graph

Content: Explain how microservices talk to each other.

Figure: Insert a diagram of a "Producer -¿ Broker -¿ Consumer" architecture here.

See Figure B.1.

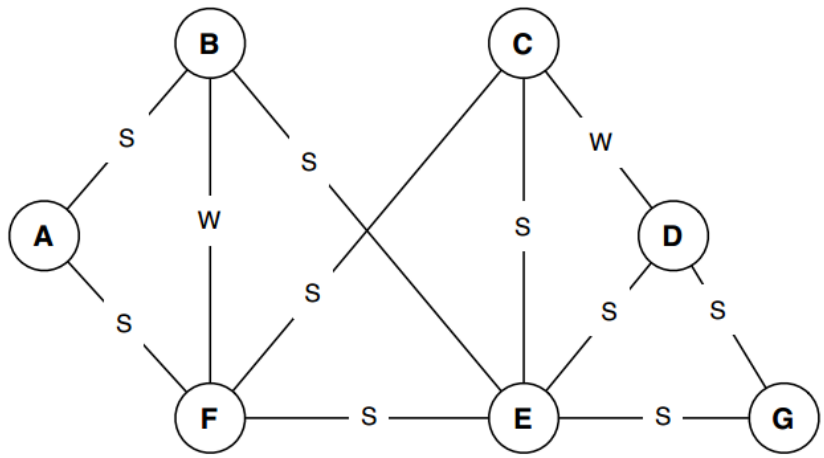


Figure B.1: A simple figure in LaTeX. Reproduced from http://tinyurl.com/nqtrlj5 with the permission of the copyright owner.

## B.2 Section Template Comparison

Content: Lorem

Table: Use the template's table code to compare them (e.g., Column A=Lorem, B=Inps, C=Lode).

| A | B | A XOR B |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

Table B.1: A simple table in LaTeX.

## B.3 Summary

Content: "Existing benchmarks are outdated. This project fills that gap."

# Appendix C

# Another Appendix

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. Nulla consequat massa quis enim. Donec pede justo, fringilla vel, aliquet nec, vulputate eget, arcu. In enim justo, rhoncus ut, imperdiet a, venenatis vitae, justo. Nullam dictum felis eu pede mollis pretium. Integer tincidunt. Cras dapibus. Vivamus elementum semper nisi. Aenean vulputate eleifend tellus. Aenean leo ligula, porttitor eu, consequat vitae, eleifend ac, enim. Aliquam lorem ante, dapibus in, viverra quis, feugiat a, tellus. Phasellus viverra nulla ut metus varius laoreet. Quisque rutrum. Aenean imperdiet. Etiam ultricies nisi vel augue. Curabitur ullamcorper ultricies nisi. Nam eget dui. Etiam rhoncus. Maecenas tempus, tellus eget condimentum rhoncus, sem quam semper libero, sit amet adipiscing sem neque sed ipsum. Nam quam nunc, blandit vel, luctus pulvinar, hendrerit id, lorem. Maecenas nec odio et ante tincidunt tempus. Donec vitae sapien ut libero venenatis faucibus. Nullam quis ante. Etiam sit amet orci eget eros faucibus tincidunt. Duis leo. Sed fringilla mauris sit amet nibh. Donec sodales sagittis magna. Sed consequat, leo eget bibendum sodales, augue velit cursus nunc.